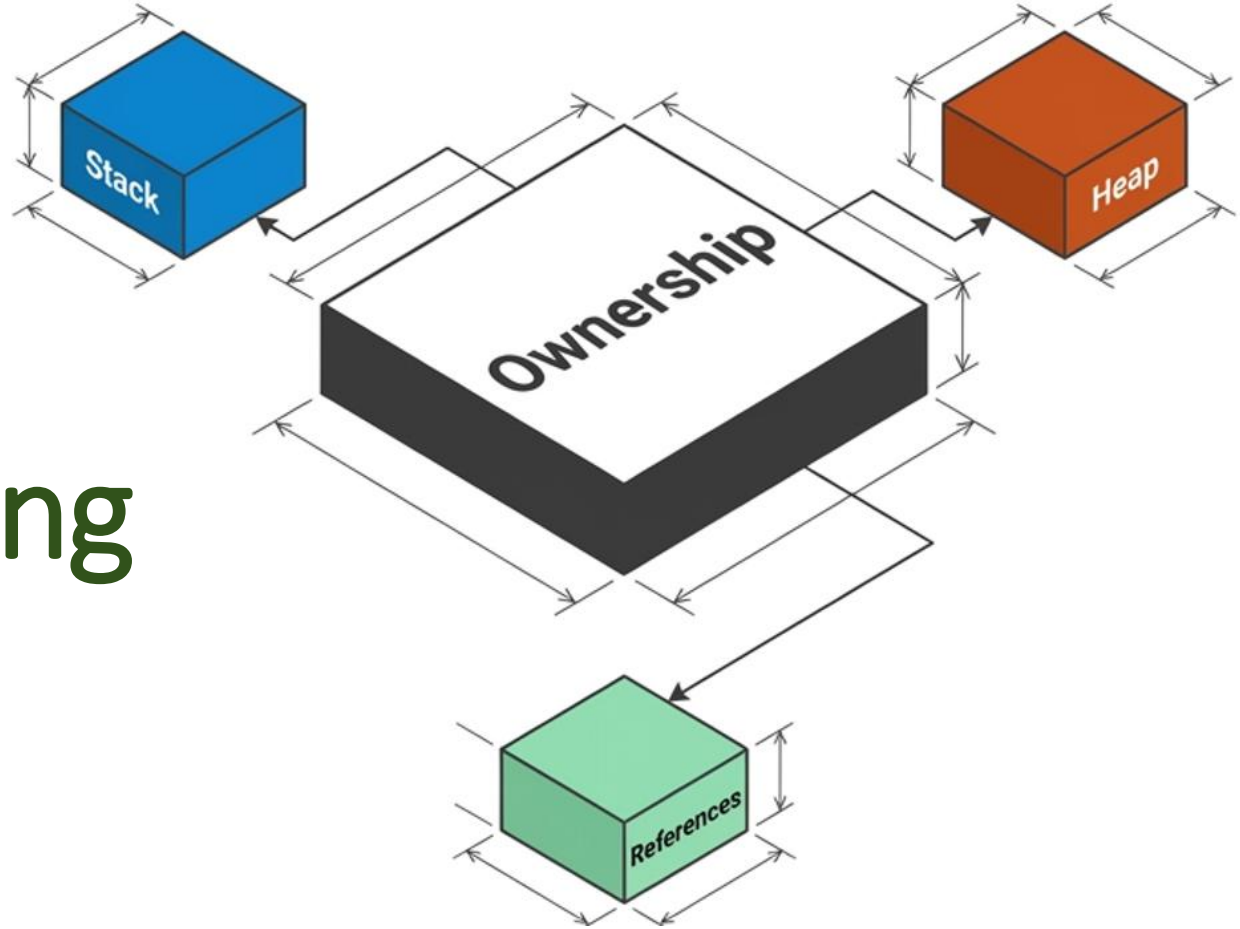
Ownership y Borrowing en Rust



La arquitectura de la memoria: máxima velocidad, cero errores y sin recolector de basura.



Gestión Manual (C, C++)

- **Mecanismo:** El programador pide y libera la memoria.
- **Problema:** Errores humanos inevitables (olvidos de liberación, doble liberación).



Garbage Collection (Java, Python)

- **Mecanismo:** Un programa automático busca y limpia datos sin uso.
- **Problema:** Consume memoria extra y causa pausas impredecibles en el rendimiento.



Ownership (Rust)

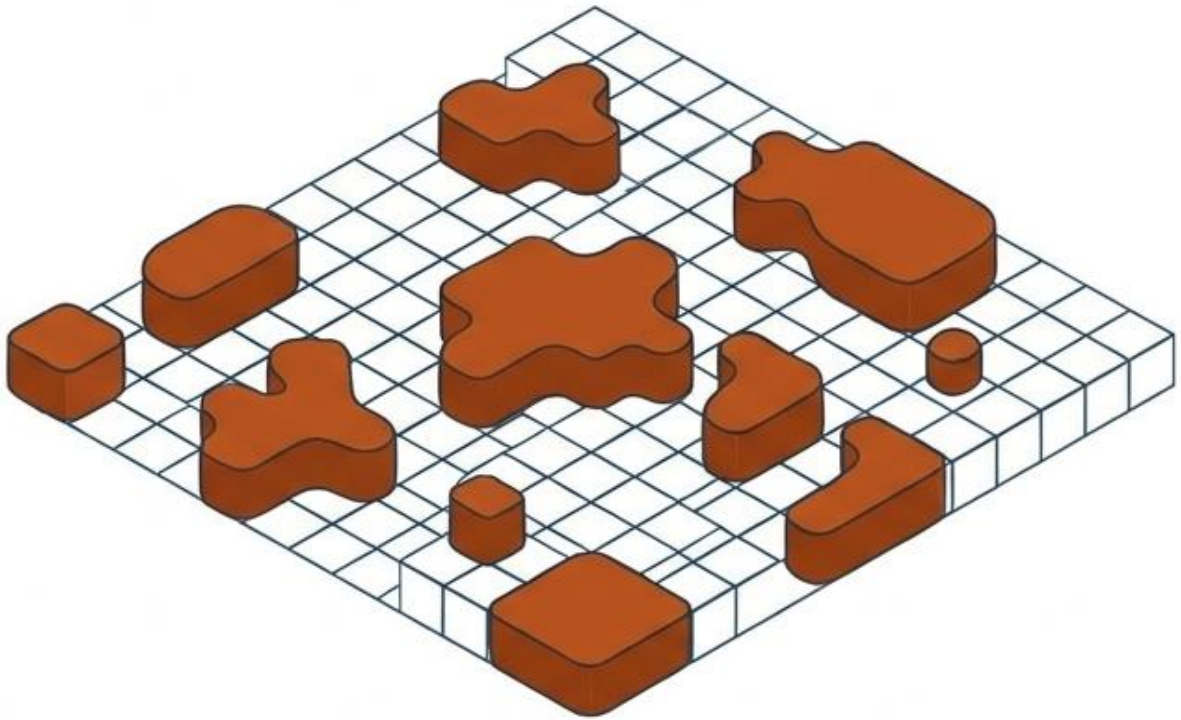
- **Mecanismo:** Reglas de propiedad verificadas por el compilador.
- **Solución:** Combina la velocidad pura de C++ con la seguridad garantizada, sin margen para el error humano.

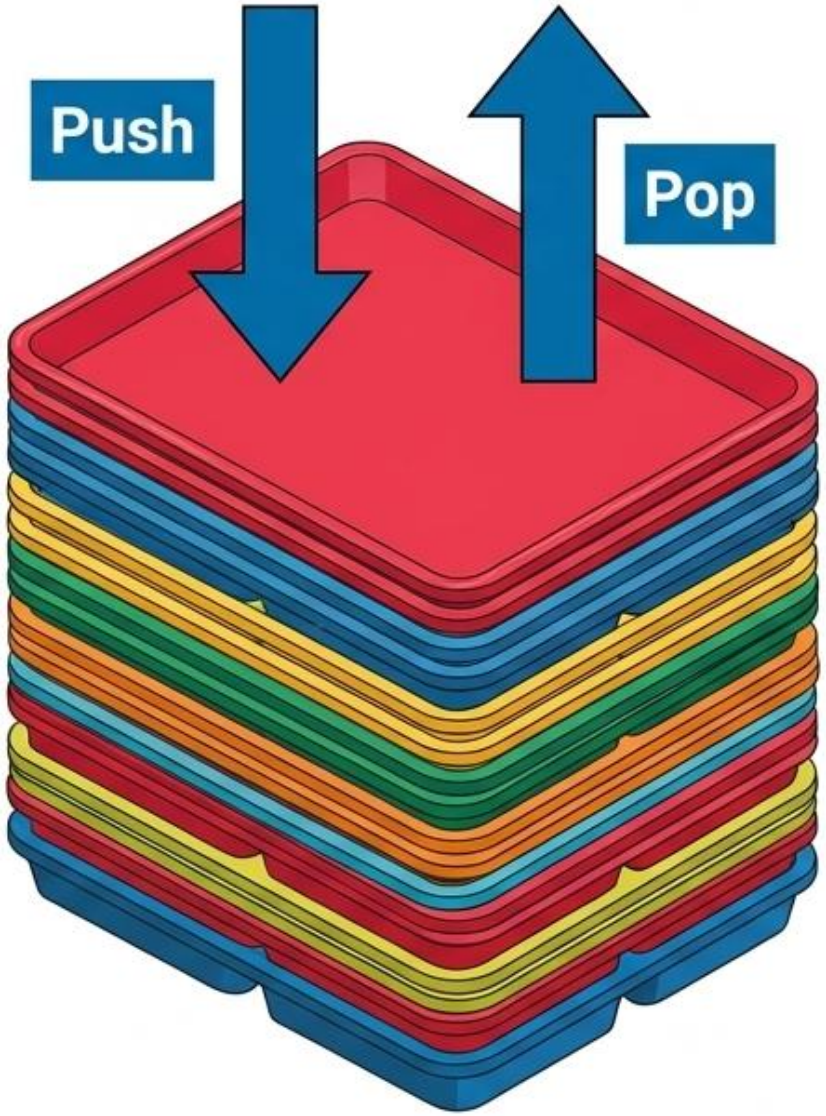
Para ejecutar un programa, Rust distribuye los datos en **dos regiones de memoria** con propósitos opuestos. El compilador decide **automáticamente qué región usar** basándose en el **tamaño del dato** y su **flexibilidad en tiempo de ejecución**.

EL STACK



EL HEAP

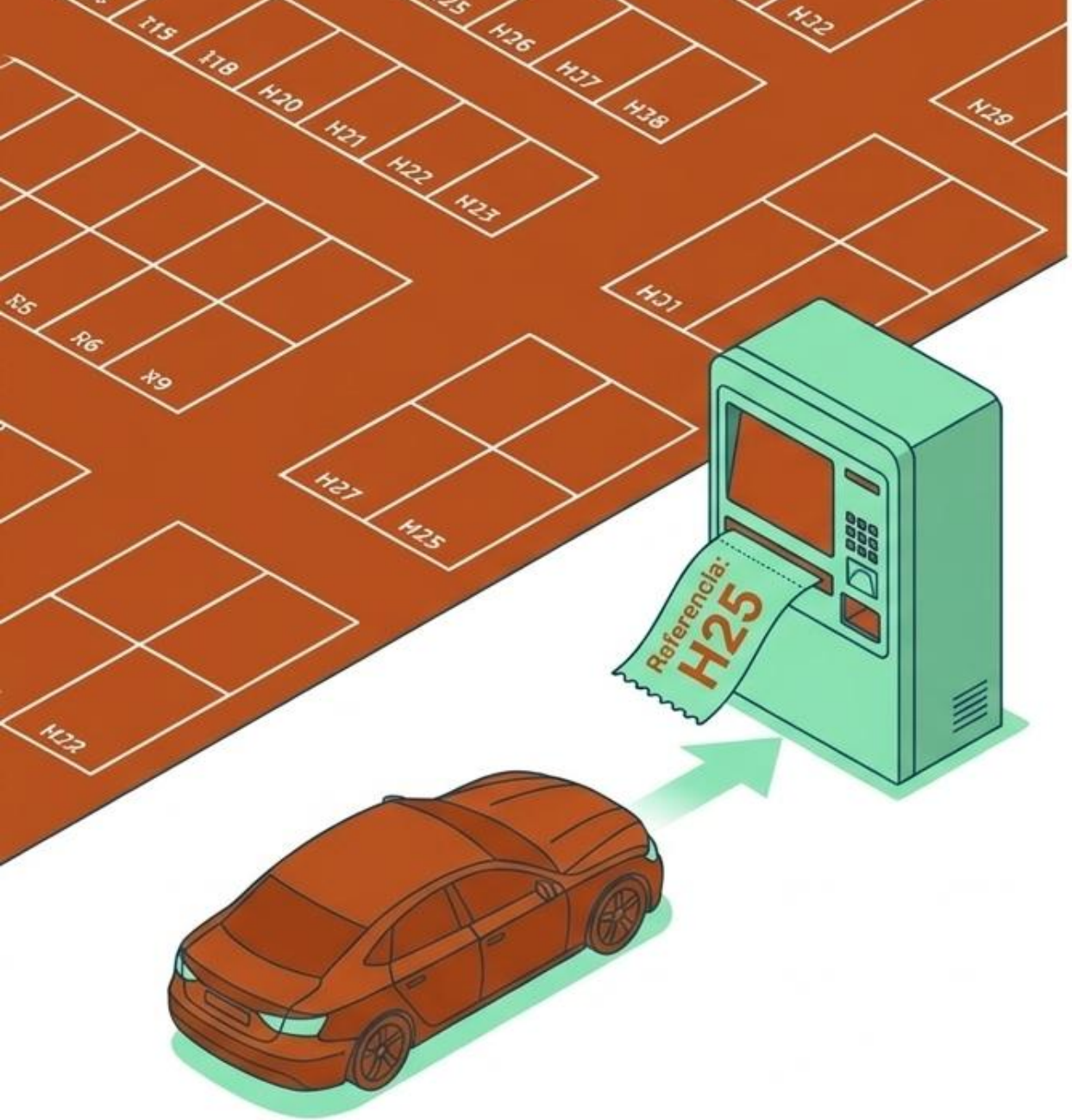




El Stack (La Pila): Orden Secuencial Estricto

- **Comportamiento (LIFO):** Último en entrar, primero en salir. La memoria se añade (push) y se retira (pop) siempre desde la cima.
- **Regla de Oro:** Todo dato en el Stack debe tener un tamaño fijo y conocido al momento de compilar (ej. números enteros, booleanos). No puede crecer ni encogerse.
- **Ventaja:** Velocidad extrema. No hay que buscar espacio disponible, los datos simplemente se apilan.

Almacena variables locales y llamadas a funciones (datos temporales). Las variables se eliminan solas cuando la función termina



El Heap (El Montículo): Espacio Dinámico

- **Comportamiento:** Para datos cuyo tamaño es impredecible (ej. texto ingresado por el usuario).

- **El Proceso de Asignación:**
  1. Rust solicita espacio dinámico.
  2. El Allocator busca un hueco libre en el almacén.
  3. El sistema devuelve una referencia (la dirección de memoria o ticket de estacionamiento).

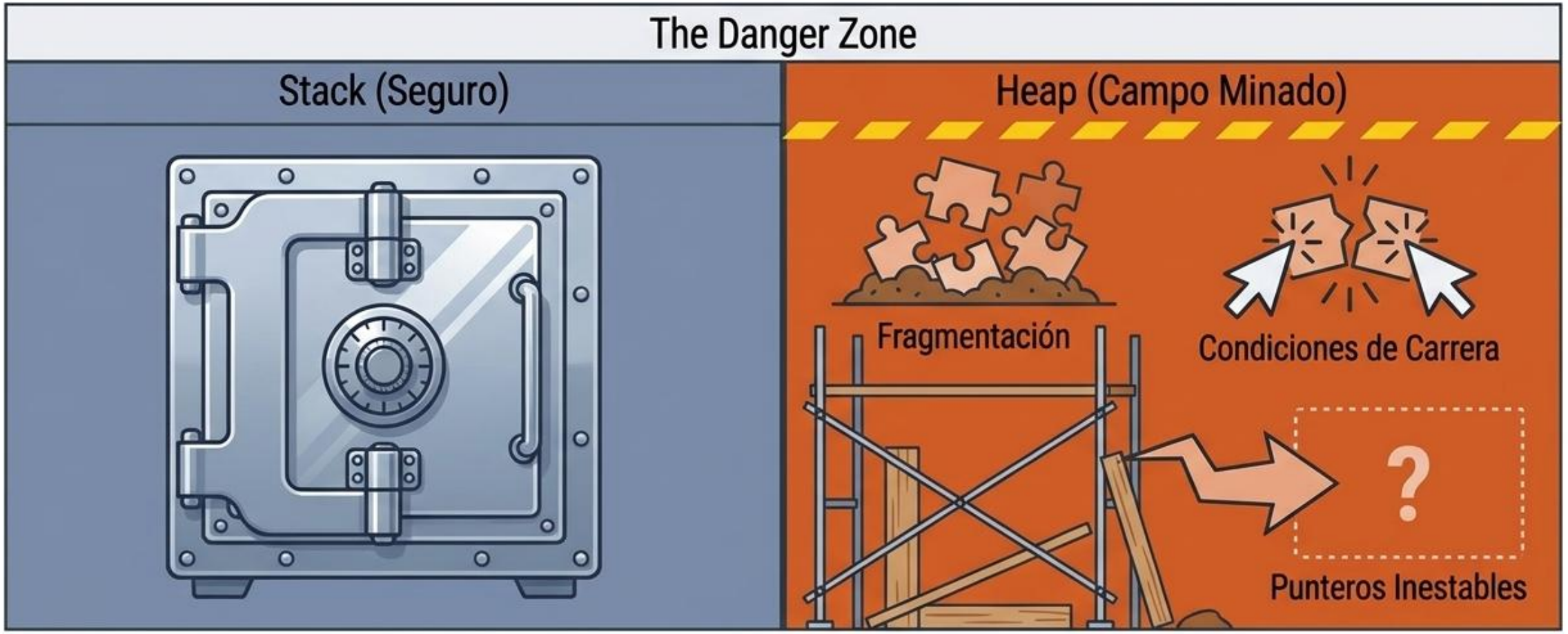
- **Compensación:** Es más lento. Requiere buscar espacio libre y el programa debe seguir la referencia para leer el dato.

Comparativa de Gestión: Stack vs. Heap

	Stack	Heap
Vida Útil	Limitada al alcance (scope).	Dinámica y prolongada.
Tamaño	Limitado y de tamaño fijo.	Flexible y variable en ejecución.
Gestión	Automática (puntero del stack).	Manual o compleja (Garbage Collector / Ownership).
Velocidad	Muy rápida (memoria contigua).	Más lenta (búsqueda y fragmentación).

El Heap es Poderoso, pero Vulnerable

El Stack es seguro por naturaleza gracias a su estructura predecible. El Heap, al carecer de este orden estricto, es un “campo minado” de complejidad:



Los Errores Fatales de la Memoria Manual



Fugas de Memoria

Consumir memoria dinámicamente y olvidar liberarla, colapsando el sistema.



Punteros Colgantes (Use-After-Free)

Intentar acceder a un bloque de memoria que ya fue vaciado o reasignado.



Doble Liberación

Ordenar al sistema que libere la misma memoria dos veces, corrompiendo la gestión del Heap.



Ownership: La Solución Revolucionaria de Rust

En lugar de depender del programador (C++) o de un lento recolector de basura (Java),
Rust protege el Heap mediante el sistema de Propiedad (Ownership):



1. Propietario Único.

Cada valor en el Heap tiene un único propietario responsable.



2. Transferencia Estricta.

La propiedad se puede transferir, pero jamás se duplica el control.



3. Liberación Automática.

Cuando el propietario sale de su alcance (scope), la memoria se libera al instante.

Ownership - propiedad

El propósito de la propiedad es asignar la responsabilidad de la reasignación de la memoria, en especial la memoria heap.

La variable “es_joven” tiene la responsabilidad de liberar la memoria de la pila al salir del ámbito.

Ambito (scope) y Ownership

Las variables existen en el ámbito en el que son definidas **{ }**

```
fn main() {
    let edad = 33;
    let es_joven = true;

    println!("{edad}");
    println!("{es_joven}");

    // variable edad existe
} // es_joven y edad están fuera de ambito (scope)
```

```
fn main() {
    let edad = 33;
    {
        let es_joven = true;
    }
    println!("{edad}");
    println!("{es_joven}"); //error, está fuera de ámbito

    // variable edad existe
}
```

Las 3 Reglas Fundamentales del Ownership

1



Cada valor en Rust tiene un único dueño (owner). Usualmente es el nombre de una variable.

2



Solo puede haber un dueño a la vez.

3



Cuando el dueño sale de su ámbito temporal (scope), el valor se elimina y la memoria se libera automáticamente.

Copiar vs. Mover: El destino al reasignar variables

```
let B = A;
```

Copy Trait
(El camino del Stack)

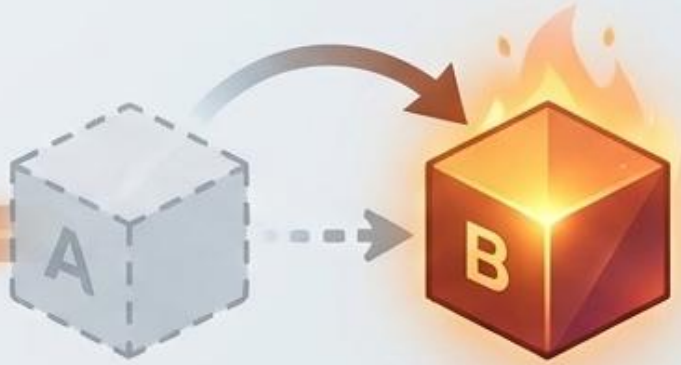




Los valores **primitivos** (enteros, booleanos) se **duplican** automáticamente.
A y **B** existen **independientemente**.

Move
(El camino del Heap)





Los tipos complejos y dinámicos transfieren la propiedad. Rust invalida la variable original para evitar la doble liberación.

Trait copy

Un trait es como un contrato que hay que cumplir. (comparable a una interfaz en Java)

El **trait copy** establece que un dato se copia y crear un duplicado completo.

Implementan el trait copy las variables almacenadas en el **Stack** que son de tipo primitivo y tienen un **tamaño fijo**.

```
fn main() {
    let tiempo = 2025;
    let año = tiempo;

    //no se cambia la propiedad porque es un dato primitivo y se copia
    println!("El tiempo es {tiempo}. Es el año {año}.");

}
```

2025
2025
Stark

Caso Práctico: ¿Piedra o Goma elástica?



String Slice (&str)

- Texto **fijo**, codificado directamente en el archivo binario.
- Tamaño **inmutable**, velocidad extrema.
- Implementa **Copy**. Se lee desde el **Stack**.



El tipo String

- Texto **dinámico** que puede crecer y modificarse.
- Vive en el **Heap**.
- No implementa **Copy**. Si se reasigna, se **mueve** el **Ownership** por completo (a menos que usemos **.clone**()).

Trait copy con cadenas

Rust tiene dos tipos de cadenas:

1. **str** que son literales de texto con tamaño fijo → let comida: &str = "pasta";

   comida es una referencia y no pasa la propiedad. let txt = "Hola Mundo"; let txt2 = txt;

2. **String** que es una cadena de texto dinámica que puede cambiar su tamaño, por ejemplo, si pedimos datos al usuario, guardamos el contenido de un archivo, … → **No imp trait copy**

```
fn main() {
    let mut text: String = String::new();
    let dulce = String::from("KitKat");

    text.push_str("Hola"); //ahora guarda "Hola"
    text.push_str(" y Adios"); //ahora guarda "Hola y Adios"
}
```

```
let persona = String::from("Juanito");
println!("Mi nombre es {persona}");

let genio = persona;

// println!("Mi nombre es {persona}"); Error, movida la propiedad
```

```
error[E0382]: borrow of moved value: `persona`
```

drop y método clone

- Cuando una variable se sale de ámbito, Rust llama automáticamente a la función **drop()**

Esta función también se puede invocar de manera manual

```
let persona = String::from("Juanito");

 drop(persona);

 // let genio = persona; //Error, no existe
```

- Rust evita hacer copias siempre que sea posible, por rapidez, menos memoria, ...

Si necesitamos duplicar los datos es necesario hacerlo explícitamente con el método **.clone()**

Método **clone**() → duplica el valor

```
let persona = String::from("Boris");
let genio = persona.clone();

println!("This is {persona}.");
```

Préstamos (Borrowing): Observar sin poseer



El problema

Mover el ownership para usar un dato en funciones es tedioso

¿Qué pasa si solo quieres leer un dato?

La solución (&)

Las **Referencias**. Permiten crear un puntero temporal hacia la dirección de memoria

El resultado

Puedes usar el dato sin quitar la propiedad al dueño original, **no ocurre un Move**

Referencia & y desreferencia *

& → guarda una dirección de memoria

***** → accede a los datos a los que apunta una referencia

```
let mi_valor_pila = 2; //dato guardado en stack
let mi_referencia_entero = &mi_valor_pila;
println!("{}", *mi_referencia_entero); //se puede quitar el *


let mi_valor_heap = String::from("Hola, caracola");
let mi_referencia_heap = &mi_valor_heap;
println!("{}", *mi_referencia_heap);  //se puede quitar el *
```

El **propietario** sigue siendo "*mi_valor_pila*"

"*mi_referencia_a_entero*" solo guarda la **dirección de memoria** donde está el dato para consultarlo

El * en los println() no es necesario porque al tener el trait **Display** se hace una **desreferencia automática**

Préstamos (Borrowing): Compartir sin Riesgos

Para no transferir la propiedad constantemente, Rust permite prestar referencias (Borrowing), bajo una regla matemática inquebrantable:

Referencias Inmutables



Puedes tener infinitas referencias de "solo lectura" simultáneas.

Referencia Mutable



Solo puedes tener una referencia de "escritura" a la vez, y no puede coexistir con ninguna de lectura.

Convivencia de las referencias

Referencias inmutables (&T)	Referencias mutables (& mut T)
Permiso para leer ✓	Permiso para leer y modificar ✓
Permiso para modificar X	
Límite simultaneo: Infinitas a la vez. (Es 100% seguro porque nadie puede alterar el dato mientras se lee)	Límite simultaneo: **¡Solo UNA a la vez!** (Rust previene las colisiones de datos o DATA RACE al nivel del compilador. Nadie más puede leer o escribir)

Múltiples referencias

Las referencias inmutables implementan el trait **copy**

Multiple referencias inmutables

```
fn main() {
    let coche = String::from(“Rojo");
    let ref1 = &coche;
    let ref2 = &coche;

    println!("{ref1} y {ref2} y {}", &coche);
}
```

¡Solo UNA referencia mutable!

```
fn main() {
    let mut coche = String::from("Red");
    let ref1 = &mut coche;
    let ref2 = &mut coche; //Error
    println!("{ref1}");
    //println!("{ref2}");
}
```

Ownership con referencias mutables e inmutables

```
//varias referencias inmutables (impl. copy)

let cafe = String::from("Moka");
let a = &cafe;
let b = &cafe; //es equivalente -> let b =  a;

println!("{a} y {b}");
```

```
//Una referencia mutable   (NO copy -> prestamos de propiedad )

  let mut cafe = String::from("Moca");
   let a = &mut cafe; //préstamo de referencia mutable
   println!("{a}");
   let b = a; //cambio de propiedad, a ya no existe

   println!("{b}");
   //el préstamo se devuelve después del último uso de variable
   println!("{cafe}");
```

Propiedad (ownership) y funciones

Cuando se pasa un argumento a una función y se almacena en un parámetro de dicha función funciona igual que en la asignación dependiendo del tipo de dato

- ✓ Datos primitivos (stack), implementan en trait **copy** → se hace una copia y no se transfiere la propiedad
- ✓ Datos del montón (heap), por ejemplo, String → se mueve la propiedad

```
fn main() {
    let cantidad = 24;
    imprime_valor(cantidad);

    let naranjas = String::from("naranjas");
    imprime_texto(naranjas);

    println!("{cantidad} si es válida");
    //println!("{naranjas} ya no exite");
}
```

```
fn imprime_valor (valor :i32){
    println!("El valor es {valor}");
}

fn imprime_texto(valor: String) {
    println!("El valor es: {valor}");
}
```

Parámetros mutables

Los **parámetros** declarados en las funciones son, por defecto, **inmutables**

⇒ Si queremos cambiar el valor del parámetro dentro de la función hay que declararlo como mut

⇒ Es independiente que la variable externa del main() sea o no mutable.

```
fn main() {
    let menu = String::from("Hamburguesa");
    agregar_papas(menu);}


fn agregar_papas(mut comida: String) {
    comida.push_str(" y Papas Fritas");
    println!("{comida}");
}
```

menu no necesita ser mutable porque no cambia. **comida** si debe ser mutable para poder concatenar el texto.

Referencias mutables e inmutable en funciones

Las referencias también pueden ser mutables e inmutables.

En este caso se está prestando la referencia mutable y cuando sale de ámbito (de la función) se devuelve el préstamo.

```
fn main() {
    let mut comida_actual = String::new();
    agregar_harina(&mut comida_actual);
    mostrar_mi_comida(&comida_actual);
}

// comida: String       // mut comida: String
// comida: &String    // comida: &mut String

fn agregar_harina(comida: &mut String) {
    comida.push_str("Agregar harina");
}

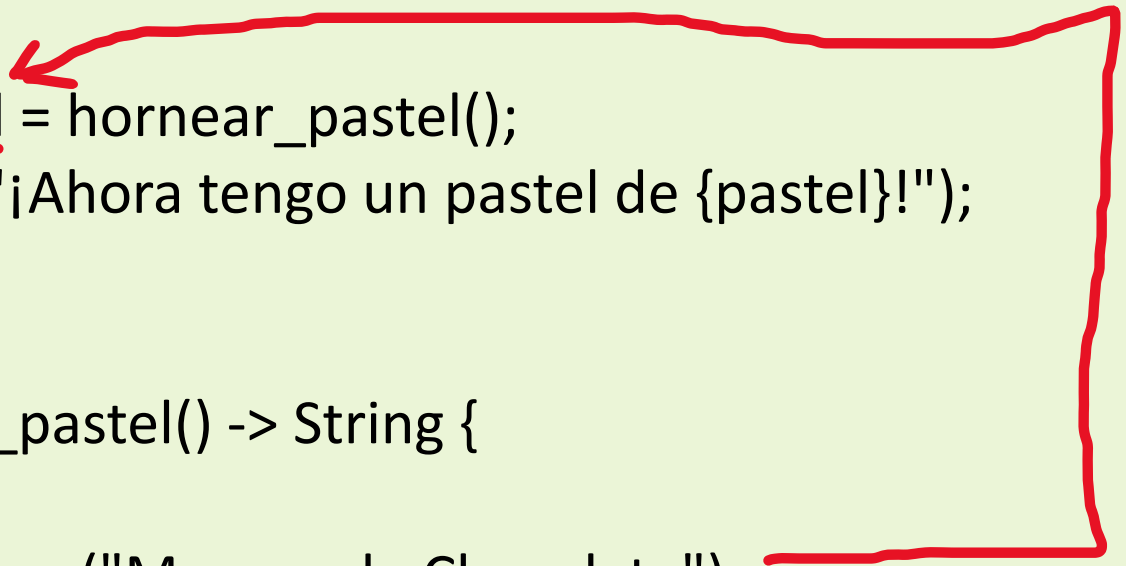
fn mostrar_mi_comida(comida: &String) {
    println!("Pasos de la comida: {comida}");
}
```

Valores de retorno

Cuando las funciones devuelven un valor, **la propiedad se desplaza** de la función hasta el main() o a donde se devuelva.

```
fn main() {
    let pastel = hornear_pastel();
    println!("¡Ahora tengo un pastel de {pastel}!");
}

fn hornear_pastel() -> String {
    // pastel
    String::from("Mousse de Chocolate")
}
```



Ownership y Borrowing en arrays y tuplas

⟹ La estructura principal es el dueño de todos sus elementos internos

⟹ Si extraes un valor **primitivo**, Rust crea automáticamente una copia (trait **copy**)

⟹ Si extraes un valor **dinámico** (**heap**), el **ownership se mueve** dejando un hueco en la estructura original

```
let inscripciones = (true, false, true);
let primera = inscripciones.0; //bool implementa el trait Copy, primera recibe una copia del valor.
println!("{primera} y {inscripciones:?}");

 let lenguajes = (String::from("Rust"), String::from("JavaScript"));
// no mueve el primer elemento, sino que crea una referencia inmutable (un préstamo)
 let primero = &lenguajes.0;
 println!("{primero} y {lenguajes:?}");
//(sin &), el String se movería, porque String no es Copy. La tupla quedaría parcialmente movida y la línea
println!("{lenguajes:?}") daría error de compilación.
```